

CIS 5550: Internet & Web Systems

Assignment 2: Dynamic Web Server

1 Overview

For this assignment, you will extend your static HTTP server from HW1 to support dynamic content and routes. The API will be based on the API from [Spark Framework](#), a real-world framework creating web applications in Java. Your solution will be used as a foundation for several of the other homework assignments, as well as for the frontend of your search engine at the end.

Spark Framework is a great way to write very compact applications. The following is a complete application that should run on your solution at the end:

```
import static cis5550.webserver.Server.*;

class HelloWorldApp {
    public static void main(String args[]) {
        port(8080);
        get("/hello/:name", (req,res) ->
            { return "Hello "+req.params("name"); } ));
    }
}
```

When this code runs and you open `http://localhost:8080/hello/Bob` in a browser window, you should see the response “Hello Bob”.

To keep the assignment manageable, we will implement a small subset of Spark Framework’s functionality: some of the functions for configuring the server and creating routes, the Request and Response objects, and the actual route-matching code. We will skip most of the complicated features, such as filters or transformers. Also, we will provide some of the boilerplate code for you (specifically, the RequestImpl object), since this is tedious to write and you wouldn’t learn much from it.

As in the earlier assignments, please do use Google, answers from the discussion group, the [Java API reference](#) and, if necessary, a good Java book to solve simple problems on your own. The [Spark Framework documentation](#) may be useful if you have questions about the API. If none of these steps solve your problem, please post on the discussion group, and we will be happy to help!

2 Requirements

Please start by downloading the HW2 package from <http://cis5550.net/hw2.zip>. This contains a README file with a short questionnaire, an Eclipse project definition (which you can ignore if you are not using Eclipse), a small test suite, three interfaces (Request, Response, and Route), and an implementation of the first interface (RequestImpl). Your solution must meet the following requirements:

Server API: You should have a class `cis5550.webserver.Server`, as in HW1. This class should have the following *static* methods:

- `port (N)` should tell the HTTP server to run on port `N`;
- `get (p, L)`, `put (p, L)`, `post (p, L)` should create GET, PUT, and POST routes, respectively; `p` is a path pattern (`String`), and `L` is a lambda that accepts a pair `(req, res)`, where `req` is an object of type `Request` and `res` is an object of type `Response`, and returns an `Object`.

The server should also contain a public static class `staticFiles` with static method `location (P)`. When this method is called with a string `P`, the server should start serving static files from path `P`, just as your HW1 solution did. The web server should start the first time `get`, `put`, `post`, or `location` is called. You may assume that `port ()` is called at most once, and that it would be the first call. If `port ()` is not called, the server should run on port 80.

Route API: The two arguments that are passed to the route handler should implement all of the functions from the `Request` and `Response` interfaces, respectively. The expected behavior is documented in `Request.java` and `Response.java`. Notice that an implementation of the `Request` object has already been provided as `RequestImpl`.

Error handling: If a route throws any exception and `write ()` has not been called, your solution should return a 500 Internal Server Error response. If `write ()` has been called, it should simply close the connection.

Miscellaneous: The intention is that you will reuse and extend your `Server` implementation from HW1; you can simply copy over your code to the HW2 project, and then remove the `main ()` method. (Recall that web applications will have their own `main ()` method, and interact with your server through calls like `get ()` and `post ()`.) As in HW1, your solution should be able to handle any headers in any order, it should send valid, correctly formatted HTTP responses, and it should handle multiple requests concurrently.

Packaging: Your solution should be in a directory called `HW2`, which should contain 1) the `README` file from the HW2 package, with all the fields filled in, and 2) a subdirectory called `src` with *all* of your source code, in the directory structure Java expects (with subdirectories for packages). Your solution *must* compile without errors if you run `javac --source-path src src/cis5550/webserver/Server.java` from within the `HW2` folder. Please do try this before you submit! Submissions that fail this basic check will receive a zero.

3 Suggested approach

We suggest that you use the steps below to solve this assignment; however, feel free to use a different approach, or to ignore this section entirely. We will give credit for your solution, as long as it meets the requirements above.

The HW2 package contains a small test suite in `cis5550.test.HW2TestClient` and `cis5550.test.HW2TestSever`, which includes a subset of the tests we will use for grading. Several of the steps below will reference these tests. To run tests, first run `HW2TestServer` (which contains a `main ()` method that defines some routes) and then, without terminating the server, run `HW2TestClient` – e.g., in a separate terminal window. As in HW1, you can run all the tests by invoking `HW2TestClient` without command-line arguments, or you can specify a list of tests to run. However, please keep in mind that *all* the features from Section 2 are required, not just the ones that are covered by the test suite!

Step #1: Copy over your HW1 code. You can simply cut and paste your `Server` class into the HW2 project. If you defined additional classes for HW1, please do not forget to copy and paste these as well.

Step #2: Add the static methods. Add a public static class `staticFiles` *within* your `Server` class, and add a public static method `location(String s)` within it. Also, in your main server class, add three public static void methods called `get`, `post`, and `put`, which should each take a `String` and an instance of `Route`, and a public static void method `port()`, which should take an `int`. Now `cis5550.test.HW2TestSever` should compile, and not complain about missing methods.

Step #3: Launch the instance on demand. Add to the `Server` class 1) a static field that can contain an instance of `Server` and is `null` initially, and 2) a static flag that is `false` initially. In each of the server API methods, check whether the field is `null`, and, if so, create a `Server` instance and put it into the field. Additionally, in methods other than `port`, check whether the flag is `false`, and if so, set it to `true` and launch a thread that executes a `run()` method in the server. (This is needed because the server will be handling requests in the background, while the application is free to do other things.) Rename your `main()` method from `HW1` to `run()`, but get rid of the arguments that specified the port number and the folder location; instead, make the `port()` and `location()` methods set fields in the server instance, and use the values from these fields. Notice that `location()` may not be called at all; your solution should start serving static files only if and when this method is called. The static test in `HW2TestClient` should now pass. (Remember that `HW2TestServer` needs to be running while you use the test client; the client will print a message to remind you.)

Step #4: Add a routing table. Add some kind of data structure that can store routes. For each route, you will need to remember three things: 1) the method (`GET`, `PUT`, or `POST`), 2) the path pattern, and 3) the handler, which will be an instance of `Route`. Change the `get()`, `put()`, and `post()` methods you created in Step #2 to add an entry to this data structure.

Step #5: Implement Response. Next, write an implementation of the `Response` interface – say, `ResponseImpl`. Most of the methods simply set values of the response, so you can have fields for the status code, the body, and the headers, and update them from the relevant methods. Keep in mind that the body may be binary, so the field for it should be a `byte[]` and not a `String`; the `body()` method can convert the string to bytes by calling `getBytes()`. Leave the `write()` method empty for now.

Step #6: Handle dynamic requests. In your `run()` method, find the point after the request has been decoded but before you check for static files. At this point, add code that iterates over the routes and looks for an entry that 1) matches the method from the current request, and 2) has a path pattern that matches the URL in the request. (If there are multiple matches, you can use any of them.) If no entries match, look for static files as before – but if a match is found, instantiate `Request` and `Response` objects (leave the named and query parameter Maps as `null` for now), and pass them to the `handle` method on the `Route`. If this throws an exception, return the 500 response as required. If not, set the `Content-Length` header and then write out the headers; then write out the body, if there is any. (Have a look at the `Response` interface for more information about what to return in the body.) Remember to skip the static-file check when a match was found, so you don't accidentally send back *both* dynamic content *and* a static file! Now you can try out your solution with the test suite; the `get`, `post`, `put`, `hdr`, and `error` tests should all pass.

Step #7: Implement write(). Now, add the `write()` method. Notice that this is supposed to write out the headers the first time it is called, so this may involve a bit of refactoring. `RequestImpl` has a reference to your `Server`, so you can call methods on that if it is useful. Be sure to keep track of whether the headers have already been written, so you don't accidentally write them out twice. Now the `write` test case should pass. Also, re-run the earlier tests to make sure that the refactoring did not break anything.

Step #8: Implement path matching. So far, your code only invokes routes when the URL matches the route's path exactly. Now it is time to add support for path parameters. Write a function that decides whether a given URL matches a given path pattern: split both URL and path pattern by forward slashes (`/`), then check whether a) both have an equal number of pieces, and b) the pieces are either identical or the

path pattern piece is a named parameter, i.e., starts with a colon (:). It is useful to generate a Map of the path parameters along the way, to avoid code duplication. Then you can simply pass this Map to the Request object, so it can return it from `params()`. Now the `pathpar` test should pass.

Step #9: Add query parameters. Now add code that looks for query parameters in both relevant places (after the ? in the URL, and in the body if the content type has the value `application/x-www-form-urlencoded`). If any are found, split them by the ampersand (&) and the pieces into name-value pairs by the equals sign (=), URL-decode the names and values (by passing them through `java.net.URLDecoder.decode()`), then put them into a Map and pass it to the Request object. Now the `qparam` check should pass as well.

Final step: Double-check the requirements! During initial development, it makes sense to focus on passing the test cases; however, please keep in mind that the test cases are not meant to be exhaustive! Once you've finished the above steps, we strongly suggest that you read over the requirements from Section 2 again and make sure that you haven't overlooked anything.

3.1 Submitting your solution

When you are ready to submit your solution, please do the following:

1. Double-check that your solution is in a `HW2` directory within your GitHub repository, and that it has the required layout (see the end of Section 2). Verify that your code compiles! Section 2 specifies the command that the autograder will run.
2. Be sure to `git add` any new files you created, then commit your changes and push them to GitHub. Please *do not* add any generated files, such as `.class` files!
3. Submit your solution on Gradescope. Choose GitHub as the submission method, pick your CIS5550 repository, and select the `main` branch.
4. Wait for the autograder to run, inspect the results, and have a closer look at any unexpected failures.

We strongly recommend that you submit your solution at least a few hours before the deadline, so there will be enough time to resolve any problems.

4 Grading

Like each of the nine homework assignments, this assignment is worth 100 points, which will be allocated as shown in Table 1.

5 Extra credit

If you like, you can implement the following additional features for some extra credit. If you do, please indicate this in the README file you submit!

Multiple hosts (+5 points): Add a public static `host()` method. An application can call `host(H)` to define routes and a static-file location for a “virtual host” `H`. When an application calls `get()`, `put()`, `post()`, or `staticFiles.location()`, these calls should apply *only* to requests whose `Host:` header is set to the argument of the most recent `host()` call (plus optionally a port number, which you should ignore); any calls before the first `host()` call should apply by default – that is, to requests whose `Host:` header does not match the argument of any `host()` calls. For instance, if the application calls

Points	Feature(s)	Test case(s)
5 points	Requesting a static file	static
15 points	GET request via a route	get
5 points	POST request via a route, with a body	post
5 points	PUT request via a route, with extra headers	puthdr
10 points	Request with path parameters	pathpar
5 points	Exception within a route	error
10 points	Using <code>write()</code>	write
10 points	Query parameters in URL and body	qparam
30 points	Other tests that we have not published	
5 points	README file completed correctly	
100 points	Total without extra credit	

Table 1: Point distribution

```
get("/foo", (req,res) -> { return "A"; });
host("bar.com");
get("/bar", (req,res) -> { return "B"; });
host("xyz.com");
get("/bar", (req,res) -> { return "C"; });
```

then a GET request for `/bar` should return B if it has a `Host: bar.com:1234` header, C if it has a `Host: xyz.com` header, and a 404 Not Found if it has a `Host: blubb.com` header. A request for `/foo` should return A with a `Host: blubb.com` header, but a 404 Not Found with `Host: bar.com:1234` or `Host: xyz.com`.

Redirection (+5 points): Implement the `redirect()` method to the response object. If the application calls `redirect(U, c)`, the server should redirect the client to URL `U`, using response code `c` (which can be 301, 302, 303, 307, or 308).

Filters (+5 points): Add two public static methods `before()` and `after()` that each take a lambda that has the same signature as a `Route` (i.e., it should accept `(request, response)` and return a `String`). The lambda(s) that are provided to `before()` should be called *before* a route is used, and the lambda(s) that are provided to `after()` should be called after the route returns; the return values should be ignored. In addition, implement the `halt(X, Y)` in the response object. When this is called while a request is at the `before()` stage, the should return an error response, with integer `X` as the status code and string `Y` as the reason phrase (Example: `halt(401, "Not allowed")`), and the relevant route should *not* be invoked.